

Main page
Table of contents
Getting involved
Wiki news
Random page

Interaction

Help
Contributing
Recent changes
Recent talks
New pages
Statistics
Requests

Tools

What links here
Related changes
Special pages
Printable version
Permanent link
Page information

In other languages

Čeština
Deutsch
Español
Français
日本語
Русский
中文 (简体)

Page Discussion

Read

View source

View history

MAC address spoofing

This article gives several methods to spoof a Media Access Control (MAC) address.

Contents [hide]

- 1 Manually
 - 1.1 [iproute2](#)
 - 1.2 [macchanger](#)
- 2 Automatically
 - 2.1 [systemd-networkd](#)
 - 2.2 [systemd-udevd](#)
 - 2.3 [systemd unit](#)
 - 2.3.1 [Creating unit](#)
 - 2.3.1.1 [iproute2](#)
 - 2.3.1.2 [macchanger](#)
 - 2.3.2 [Enabling service](#)
 - 2.4 [netctl interfaces](#)
 - 2.5 [NetworkManager](#)
 - 2.6 [wpa_supplicant](#)
 - 2.7 [iwd](#)
- 3 [Troubleshooting](#)
 - 3.1 [Connection to DHCPv4 network fails](#)
- 4 [See also](#)

Manually

There are two methods for spoofing a MAC address: [installing](#) and configuring either [iproute2](#) or [macchanger](#). Both of them are outlined below.

iproute2

First, you can check your current MAC address with the command:

```
# ip link show interface
```

where `interface` is the name of your [network interface](#).

The section that interests us at the moment is the one that has "link/ether" followed by a 6-byte number. It will probably look something like this:

```
link/ether 00:1d:98:5a:d1:3a
```

The first step to spoofing the MAC address is to bring the network interface down. It can be accomplished with the command:

```
# ip link set dev interface down
```

Next, we actually spoof our MAC. Any hexadecimal value will do, but some networks may be configured to refuse to assign IP addresses to a client whose MAC does not match up with any of known vendors. Therefore, unless you control the network(s) you are connecting to, use MAC prefix of any real vendor (basically, the first three bytes), and use random values for next three bytes. For more information please read [Wikipedia:Organizationally unique identifier](#).

To change the MAC, we need to run the command:

```
# ip link set dev interface address XX:XX:XX:XX:XX:XX
```

Where any 6-byte value will suffice for `XX:XX:XX:XX:XX:XX`.

The final step is to bring the network interface back up. This can be accomplished by running the command:

```
# ip link set dev interface up
```

If you want to verify that your MAC has been spoofed, simply run `ip link show interface` again and check the value for 'link/ether'. If it worked, 'link/ether' should be whatever address you decided to change it to.

macchanger

Another method uses [macchanger](#) (a.k.a., the GNU MAC Changer). It provides a variety of features such as changing the address to match a certain vendor or completely randomizing it.

Install the package [macchanger](#).

The spoofing is done on per-interface basis, specify [network interface](#) name as `interface` in each of the following commands.

The MAC address can be spoofed with a fully random address:

```
# macchanger -r interface
```

To randomize only device-specific bytes of current MAC address (that is, so that if the MAC address was checked it would still register as being from the same vendor), you would run the command:

```
# macchanger -e interface
```

To change the MAC address to a specific value, you would run:

```
# macchanger --mac=XX:XX:XX:XX:XX:XX interface
```

Where `XX:XX:XX:XX:XX:XX` is the MAC you wish to change to.

Finally, to return the MAC address to its original, permanent hardware value:

```
# macchanger -p interface
```

Note: A device cannot be in use (connected in any way or with its interface up) while the MAC address is being changed.

Automatically

systemd-networkd

systemd-networkd supports MAC address spoofing via [link files](#) (see [systemd.link\(5\)](#) for details).

To set a static spoofed MAC address:

```
/etc/systemd/network/00-default.link

[Match]
MACAddress=original MAC

[Link]
MACAddress=spoofed MAC
NamePolicy=kernel database onboard slot path
```

To randomize the MAC address on every boot, set `MACAddressPolicy=random` instead of `MACAddress=spoofed MAC`.

systemd-udev

Udev allows you to perform MAC address spoofing by creating udev rules. Use `address` attribute to match the correct device by its original MAC address and change it using the `ip` command:

```
/etc/udev/rules.d/75-mac-spoof.rules

ACTION=="add", SUBSYSTEM=="net", ATTR{address}=="XX:XX:XX:XX:XX:XX", RUN+="/usr/bin/ip link set dev $name address YY:YY:YY:YY:YY:YY"
```

where `XX:XX:XX:XX:XX:XX` is the original MAC address and `YY:YY:YY:YY:YY:YY` is the new one.

systemd unit

Creating unit

Below you find two examples of **systemd** units to change a MAC address at boot, one sets a static MAC using `ip` and one uses `macchanger` to assign a random MAC address. The `systemd network-pre.target` is used to ensure the MAC is changed before a network manager like [Netctl](#) or [NetworkManager](#), [systemd-networkd](#) or [dhcpcd](#) service starts.

iproute2

systemd unit setting a predefined MAC address:

```
/etc/systemd/system/macspoof@.service

[Unit]
Description=MAC Address Change %I
Wants=network-pre.target
Before=network-pre.target
BindsTo=sys-subsystem-net-devices-%i.device
After=sys-subsystem-net-devices-%i.device

[Service]
Type=oneshot
ExecStart=/usr/bin/ip link set dev %i address 36:aa:88:c8:75:3a
ExecStart=/usr/bin/ip link set dev %i up

[Install]
WantedBy=multi-user.target
```

macchanger

systemd unit setting a random address while preserving the original NIC vendor bytes. Ensure that [macchanger](#) is [installed](#):

```
/etc/systemd/system/macspoof@.service

[Unit]
Description=macchanger on %I
Wants=network-pre.target
Before=network-pre.target
BindsTo=sys-subsystem-net-devices-%i.device
After=sys-subsystem-net-devices-%i.device

[Service]
ExecStart=/usr/bin/macchanger -e %I
Type=oneshot
```

```
[Install]
WantedBy=multi-user.target
```

A full random address can be set using the `-r` option, see [#macchanger](#).

Enabling service

Append the desired network interface to the service name (e.g. `eth0`) and **enable** the service (e.g. `macspoof@eth0.service`).

Reboot, or stop and start the prerequisite and requisite services in the proper order. If you are in control of your network, verify that the spoofed MAC has been picked up by your router by examining the static, or DHCP address tables within the router.

netctl interfaces

You can use a **netctl hook** to run a command each time a netctl profile is re-/started for a specific network interface. Replace `interface` accordingly:

```
/etc/netctl/interfaces/interface

#!/usr/bin/env sh
/usr/bin/macchanger -r interface
```

Make the script executable:

```
chmod +x /etc/netctl/interfaces/interface
```

Source: [akendo.eu](#)

NetworkManager

See [NetworkManager#Configuring MAC address randomization](#).

wpa_supplicant

wpa_supplicant can use random MAC address for each ESS connection(AP) (see [\[1\]](#) for details).

Add this to your configuration:

```
/etc/wpa_supplicant/wpa_supplicant-wlan0.conf

mac_addr=1
preassoc_mac_addr=1
gas_rand_mac_addr=1
```

iwd

To randomize the MAC address when iwd starts (see [iwd.config\(5\)](#) for details):

```
/etc/iwd/main.conf

[General]
AddressRandomization=once
AddressRandomizationRange=nic
```

Specifying `AddressRandomizationRange` enables control over which part of the address is randomized. If set to `nic`, only the NIC specific octets (last three octets) are randomized. The permanent mac address of the network interface is used for the initial 3 octets. If set to `full`, all six octets of the address are randomized.

Troubleshooting

Connection to DHCPv4 network fails

If you cannot connect to a DHCPv4 network and you are using dhcpcd, which is the default for NetworkManager, you might need to [modify the dhcpcd configuration](#) to obtain a lease.

See also

- [Wikipedia:MAC spoofing](#)
- [Macchanger GitHub page](#)
- [Article on DebianAdmin](#) with more *macchanger* options

Categories: [Network configuration](#) | [Security](#)

This page was last edited on 13 June 2021, at 08:58.

Content is available under [GNU Free Documentation License 1.3](#) or later unless otherwise noted.

[Privacy policy](#) [About ArchWiki](#) [Disclaimers](#)